

Teman Tanyamu (dash_ai_me)

Lightweight self-hosted AI chat web app. An Open WebUI alternative with **under 100 MB RAM**. No Docker. No build step. One Node.js process + one SQLite file.

Stack	Node.js + Express 5 + better-sqlite3 + vanilla JS
AI Backend	9Router (http://localhost:20128/v1 , OpenAI-compatible)
Providers	AG, Groq, BAI, OpenRouter, Nvidia, Cline — 497+ models
Frontend	index.html + style.css + app.js , zero framework, dark theme
Language	English (default) / Indonesian — runtime toggle
Version	v1.0-beta
License	Private / personal use

Table of Contents

- 1. [Features](#)
- 2. [System Architecture](#)
- 3. [Request Flow Diagrams](#)
- 4. [Requirements](#)
- 5. [Installation](#)
- 6. [Management Script](#)
- 7. [Configuration](#)
- 8. [API Reference](#)
- 9. [Database Schema](#)
- 10. [Access from Other Devices](#)
- 11. [Performance](#)
- 12. [Security Notes](#)
- 13. [Troubleshooting](#)
- 14. [Changelog](#)

Features

Chat

Feature	Description
Real-time streaming	SSE relay, word-by-word from 9Router
Multi-session history	Per-user isolation in SQLite
Chat memory	Token-budget context window (default 1600 tokens ≈ 15-30 msgs, oldest auto-compressed) + rolling background summary of older turns
Cross-chat memory	Auto-extracted personal facts per user (max 20), injected into every chat; editable in Settings → Memory
Token saver	max_tokens reply cap (default 1024), history budget, per-chat Lean ⚡ toggle (answers without history)
Per-reply usage badge	prompt / completion / total tokens shown under each AI reply

Feature	Description
Rename / Delete chats	Via sidebar hover buttons; rename uses custom in-app dialog (bilingual, no native prompt)
Chat search	Instant client-side filter by title (sidebar)
Markdown + code blocks	With streaming auto-close fence fix
Copy & Retry	Per-message actions
Stop generation	Abort mid-stream
Provider timeout	Admin-configurable 30-600 s (default 120 s), applies to all upstream calls
Typing indicator	Animated dots while waiting
Temporary Chat	Zero history saved (endpoint: <code>/api/temp-chat</code>); honours Lean mode (skip history + memory = zero context)
Realtime quota UI	Sidebar quota + admin dashboard refresh automatically after every reply (dashboard auto-polls every 5 s while open)

Users & Access

Feature	Description
Login	scrypt per-user salt (<code>scrypt1:salt:hash</code>), timing-safe compare; legacy static-salt hashes auto-upgrade on login
Session purge	Expired sessions deleted on every login
Roles	<code>admin</code> (full control) and <code>user</code> (chat only)
Daily quota	Per-user limit (default 50/day), admin unlimited; quota bar shown in sidebar with model chip
User management	Admin: add / delete / reset password / activate / deactivate (self-deletion & self-deactivation blocked server-side)
Instant session revoke	Deactivate or password reset kills sessions immediately
Quota edit & reset	Admin edits any user's daily quota (1-999999) and resets one or all users' usage

Models

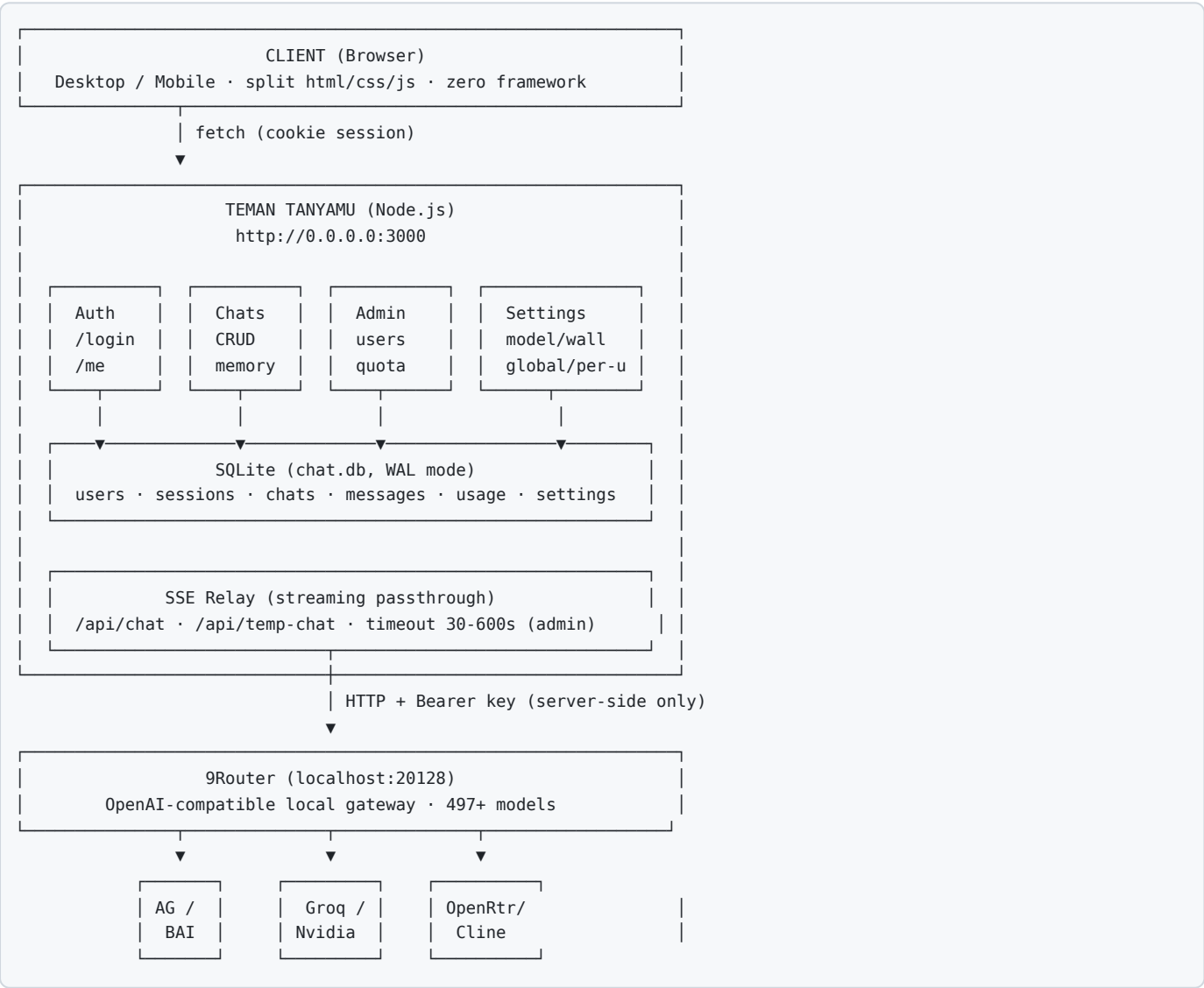
Feature	Description
Auto-fetch	Model list pulled live from 9Router
Searchable picker	Custom combobox with instant filter + keyboard nav (<code>↑/↓/Enter/Esc</code>), works for admin & locked users
Global default	Admin setting, applies to all users
Per-user override	Admin sets specific model per user
Per-chat model	Each chat remembers its model
Locked for users	Regular users cannot change model (server-enforced); UI shows a red "Terkunci / Locked" chip

Appearance

Feature	Description
Dark theme	Open WebUI inspired palette
Chat wallpaper	6 presets + custom image upload (per-user, optional global); dark overlay only for image wallpapers, gradient presets render as-is
Custom avatar	Auto-downscaled upload, strict base64 validation; rendered in chat bubbles, sidebar and admin list

Feature	Description
Global assistant avatar	Admin uploads one AI avatar for everyone (/api/admin/assistant-avatar), or resets to default mask icon
Admin avatar popover	Click any avatar row in the admin user table → upload / reset / cancel (no native prompts anywhere)
Sidebar minimize	Desktop: icon rail (user-row collapses to avatar + stacked actions), state persisted · Mobile: closes the drawer
Bilingual UI	English / Indonesian, all strings translated — dialogs, toasts, admin panel, error codes
Custom dialogs	All confirm/rename/quota dialogs are in-app modals (i18n + consistent style), never native alert/confirm/prompt
Responsive	Mobile drawer sidebar, 44px+ tap targets, admin tables become card stacks, verified 1920→320 px with no horizontal overflow

System Architecture



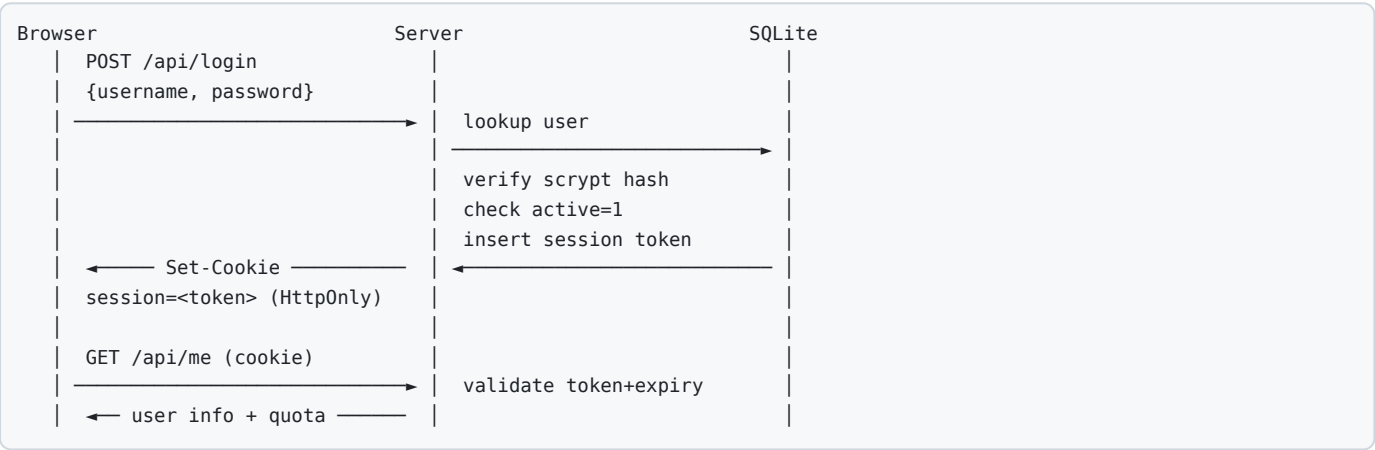
Key Design Decisions

Decision	Reason
Single file backend (server.js)	Easy audit, zero complexity

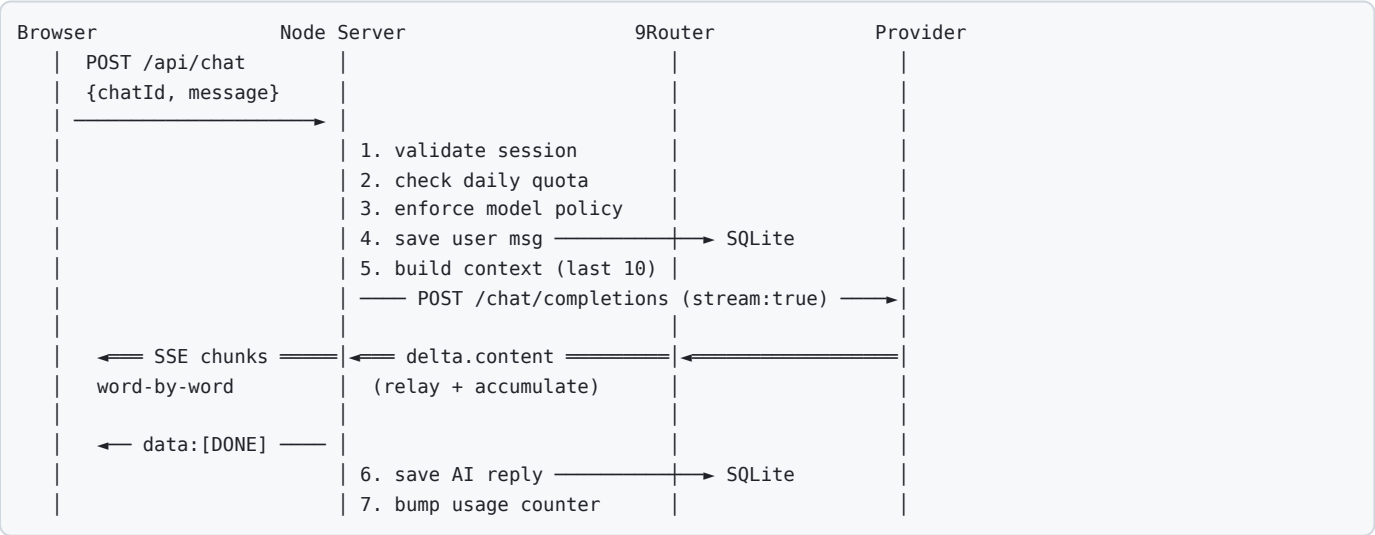
Decision	Reason
SQLite over Postgres	Zero-config, WAL mode, single file backup
Server-side API key proxy	Key never reaches the browser
SSE relay (not direct)	Streaming works cross-device, CORS-free
Error codes (ERR_*)	Translated client-side per language
no-cache delivery + versioned assets	HTML: no-store · CSS/JS: no-cache + ?v= query (updates always fresh, stale caches bust on release)
Split frontend (style.css + app.js)	Cacheable assets, lintable JS, CSP-ready

Request Flow Diagrams

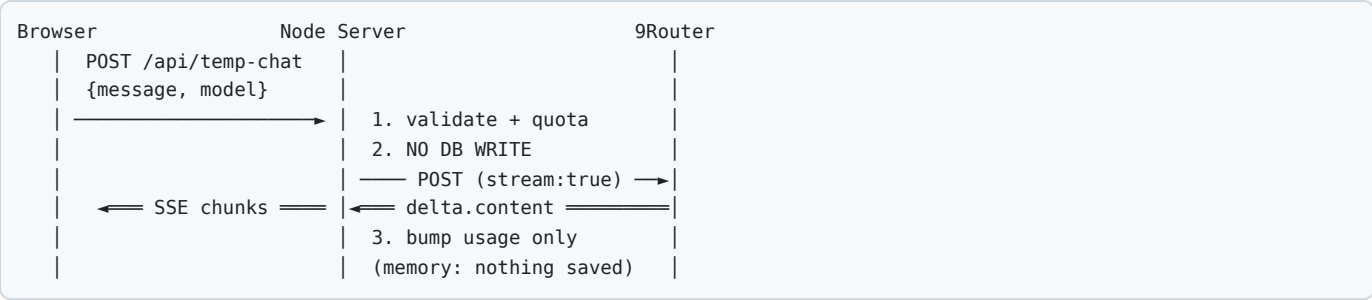
1. Login Flow



2. Streaming Chat Flow



3. Temporary Chat Flow



4. Admin Model Policy

User (non-admin) changes model?

- UI: selector disabled
- Server (double check):
 - PUT /api/me/model → 403 ERR_ADMIN_ONLY
 - PATCH /api/chats model → 403 ERR_ADMIN_ONLY
 - POST /api/chats {model} → forced to resolveUserModel()
 - POST /api/chat → chat.model auto-corrected

Admin sets global default → settings.default_model
Admin sets user override → users.model_override

Resolution order: user override > global default

Requirements

Component	Minimum
OS	Linux (tested: Ubuntu 24.04)
Node.js	>= 20 (tested: v24.21)
9Router	Running, OpenAI-compatible, default port 20128
Disk	~50 MB (incl. node_modules)

Installation

```
cd dash_ai_me
npm install      # express + better-sqlite3
node server.js  # http://localhost:3000
```

First run creates the admin account automatically:

```
username: admin
password: admin123      # change immediately via UI
```

Management Script (`manage.sh`)

All-in-one service manager (systemd user service):

```
./manage.sh start      # Start service
./manage.sh stop       # Stop service
./manage.sh restart    # Restart service
./manage.sh status     # Status, port, RAM
./manage.sh port <1-65535> # Change port (persistent, auto-restart)
./manage.sh clear-cache # Clean SQLite WAL + npm cache + old journal
./manage.sh enable-boot # Auto-start at boot (enables linger)
./manage.sh disable-boot # Disable boot autostart
./manage.sh logs [n]   # Last n log lines (default 50)
./manage.sh password   # Reset admin password to admin123
./manage.sh help       # Show help
```

`enable-boot` `RUNS` `logindctl enable-linger` so the service runs without an active desktop session.

Configuration

Environment Variables

Variable	Default	Description
<code>PORT</code>	<code>3000</code>	Web server port
<code>ROUTER_BASE</code>	<code>http://localhost:20128/v1</code>	9Router API base URL
<code>ROUTER_KEY</code>	<code>sk-a777...</code>	9Router API key
<code>ADMIN_PASSWORD</code>	<code>admin123</code>	Initial admin password (first run only)

Example:

```
PORT=8080 ROUTER_BASE=http://192.168.1.10:20128/v1 node server.js
```

File Locations

Path	Description
<code>chat.db</code>	SQLite database (WAL mode)
<code>public/index.html</code>	Struktur halaman SPA (login + app shell)
<code>public/style.css</code>	Seluruh styling frontend
<code>public/app.js</code>	Seluruh logika frontend
<code>server.js</code>	Backend (single file)
<code>.port</code>	Persisted port number (created by <code>manage.sh port</code>)
<code>~/.config/systemd/user/dash-ai-me.service</code>	systemd unit

API Reference

All endpoints require the session cookie except `/api/login` and `/api/health`. Errors return `{"error": "ERR_*"}` codes, translated client-side.

Auth

Method	Endpoint	Body	Description
POST	/api/login	{username, password}	Sign in
POST	/api/logout	-	Destroy session
GET	/api/me	-	User info + quota + effective model + global assistant avatar
GET	/api/health	-	Liveness + DB check (no auth, for uptime monitors)

Account

Method	Endpoint	Body	Description
PUT	/api/me/password	{current, next}	Change own password
PUT	/api/me/avatar	{avatar: dataURL null}	Set/remove avatar
PUT	/api/me/model	{model}	Own model override (admin)
DELETE	/api/me/model	-	Clear override (admin)

Chats

Method	Endpoint	Body	Description
GET	/api/chats	-	List own chats
POST	/api/chats	{title?, model?}	Create chat
GET	/api/chats/:id	-	Chat + full messages
PATCH	/api/chats/:id	{title?, model?}	Rename (model change = admin)
PATCH	/api/chats/:id/lean	{lean: bool}	Toggle lean (no-history) token saver per chat
DELETE	/api/chats/:id	-	Delete chat + messages

Memory (cross-chat, per user)

Method	Endpoint	Body	Description
GET	/api/memory	-	List own memory facts + enabled flag
PUT	/api/memory	{facts: string[]}	Replace full fact list (max 20, 240 chars each)
DELETE	/api/memory/:factId	-	Delete one fact
POST	/api/memory/clear	-	Wipe own memory

Chat & Preferences

Method	Endpoint	Body	Description
GET	/api/models	-	Model list from 9Router
POST	/api/chat	{chatId, message}	Streamed chat (SSE); token-budget context + system memory injected; final usage event with prompt/completion split
POST	/api/temp-chat	{message, model?, lean?}	Streamed temp chat (SSE, not saved); lean:true = zero context (no system prompt/memory)
GET	/api/wallpaper	-	Effective wallpaper
PUT	/api/wallpaper	{wallpaper null}	Own wallpaper

Admin

Method	Endpoint	Body	Description
GET	/api/admin/users	-	Users + today's usage + effective model + me id
POST	/api/admin/users	{username, password, role, daily_quota}	Create user
DELETE	/api/admin/users/:id	-	Delete user (self-delete blocked)
PUT	/api/admin/users/:id/password	{password}	Reset password (revokes sessions)
PUT	/api/admin/users/:id/active	{active: bool}	Activate / deactivate (self-deactivate blocked)
PUT	/api/admin/users/:id/quota	{daily_quota}	Edit daily quota (1-999999)
PUT	/api/admin/users/:id/reset-quota	-	Reset one user's daily usage
PUT	/api/admin/reset-usage	-	Reset all users' usage (DELETE FROM usage)
PUT	/api/admin/users/:id/model	{model\ null}	Per-user model override
POST	/api/admin/avatar	{userId, avatar\ null}	Upload / reset any user's avatar (self-sync immediate)
POST	/api/admin/assistant-avatar	{avatar\ null}	Global assistant (AI) avatar for all users
GET	/api/admin/settings	-	Global settings + today's token totals (prompt/completion)
PUT	/api/admin/settings	{default_model?, history_token_budget?, max_reply_tokens?, memory_enabled?, timeout_ms?}	Update settings (budget 200-32000, reply cap 64-8192, timeout 30-600 s)
PUT	/api/admin/wallpaper	{wallpaper\ null}	Set global wallpaper

Error Codes

Code	Meaning
ERR_UNAUTH	Not signed in
ERR_ADMIN_ONLY	Admin-only action
ERR_FIELDS_REQUIRED	Missing fields
ERR_LOGIN	Wrong credentials
ERR_INACTIVE	Account deactivated
ERR_PW_SHORT	Password under 4 chars
ERR_OLD_PW	Old password wrong
ERR_QUOTA	Daily quota exceeded
ERR_RATE_LIMITED	Too many failed logins: 5 attempts, then 2-minute IP block
ERR_NOT_FOUND	Resource not found
ERR_USERNAME_TAKEN	Username exists
ERR_TOO_BIG	Image too large
ERR_AVATAR_FORMAT	Avatar not an image
ERR_WALLPAPER_FORMAT	Wallpaper format invalid
ERR_SELF_DELETE	Cannot delete own account
ERR_SELF_DEACTIVATE	Cannot deactivate own account

Code	Meaning
ERR_BAD_BODY	Malformed JSON body
ERR_BAD_BUDGET	History budget out of range (200–32000)
ERR_BAD_MAXTOK	Reply token cap out of range (64–8192)
ERR_BAD_TIMEOUT	Upstream timeout out of range (30–600 s)
ERR_ROUTER_DOWN	9Router unreachable
ERR_TIMEOUT	Upstream model call timed out

Database Schema

```

users      (id, username, password_hash, role, daily_quota,
            model_override, avatar, active, created_at)
-- password_hash: "scrypt1:<salt>:<hash>" (per-user salt);
-- legacy static-salt hex auto-upgrades on next login
sessions   (token, user_id, expires_at)           -- expired rows purged on login
chats      (id, user_id, title, model, lean,
            created_at, updated_at)               -- lean: 1 = no-history token saver
messages   (id, chat_id, role, content, tokens, created_at)
usage      (user_id, date, request_count, tokens_used) -- PK (user_id, date)
memories   (id, user_id, content, source, updated_at) -- cross-chat facts (max 20/user)
settings   (key, value)
-- keys: default_model, wallpaper, assistant_avatar,
--       user:<id>:wallpaper, history_token_budget, max_reply_tokens,
--       memory_enabled, timeout_ms, memory_sync_chat, memory:last:<id>

```

Access from Other Devices

Server binds to all interfaces. From phones / LAN devices:

```
http://<laptop-ip>:3000      e.g. http://20.20.100.6:3000
```

For access over the internet, a Cloudflare Tunnel works out of the box: `trust proxy` is enabled and `cf-connecting-ip` is honored for login rate limiting; the session cookie gets the `Secure` flag automatically when the request arrives via HTTPS.

Performance

Metric	Measured
RAM idle	20-30 MB
RAM peak (streaming)	50-65 MB
CPU idle	~0%
Boot time	< 3 s
Streaming first token	~0.5-2 s (provider dependent)

Security Notes

- Passwords: scrypt with per-user random salt, stored as `scrypt1:<salt>:<hash>`, verified with `crypto.timingSafeEqual`; legacy static-salt hashes upgrade automatically on the next successful login
- Sessions: HttpOnly cookie, SameSite=Lax, 7-day expiry; expired rows purged on login
- **Secure cookie flag**: auto-enabled when behind HTTPS (Cloudflare Tunnel sets `X-Forwarded-Proto`)
- **Login rate limiting**: 5 failed attempts per IP, then a 2-minute block (`ERR_RATE_LIMITED`)
- `trust proxy` enabled: client IP correctly detected behind Cloudflare (`CF-Connecting-IP`)
- **Security headers** on every response: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`
- **XSS hardening**: avatar & wallpaper validated server-side against strict formats (base64 image types / `linear-gradient(...)` / `#hex` only — no `svg+xml`, no attribute breakout); all user-supplied values escaped at render
- 9Router API key never sent to browser (server-side proxy only)
- Per-user chat isolation enforced in every SQL query
- Model policy enforced server-side (not just hidden in UI)
- Quota checked before every AI call; failed/aborted streams do not consume quota
- Deactivate / password reset revokes all sessions instantly
- Caching: HTML `no-store`, CSS/JS `no-cache` + `?v=` asset version in `index.html` (bump on each release so old client caches invalidate automatically)

Troubleshooting

Problem	Fix
"9Router offline"	Check: <code>curl http://localhost:20128/v1/models -H "Authorization: Bearer <key>"</code>
Forgot admin password	<code>./manage.sh password</code>
Port already in use	<code>./manage.sh port 3001</code>
Not auto-starting at boot	<code>./manage.sh enable-boot</code> , verify <code>loginsctl show-user \$USER \\\ grep Linger</code>
Stale UI after update	Hard refresh: <code>Ctrl+Shift+R</code> (HTML/CSS/JS are all no-cache; rarely needed)
Full reset (nuclear)	<code>./manage.sh stop && rm -f chat.db* && ./manage.sh start</code> — all data gone